TESI DI LAUREA MAGISTRALE

# Implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche


CANDIDATO **Simone Vaiasinni**

RELATORE Prof. Nicola Capodieci

CORRELATORE Prof. Fabio Pellacini



**Laurea Magistrale in Informatica**

Università degli Studi di Modena e Reggio Emilia · A.A. 2025/2026

# Introduzione: dal CAD 2D al modello 3D

### Obiettivo

Da una planimetria CAD di una stanza o di un appartamento generare un **modello 3D** e renderizzarlo in **Blender** con materiali e asset.

### La sfida

L'algoritmo deve:

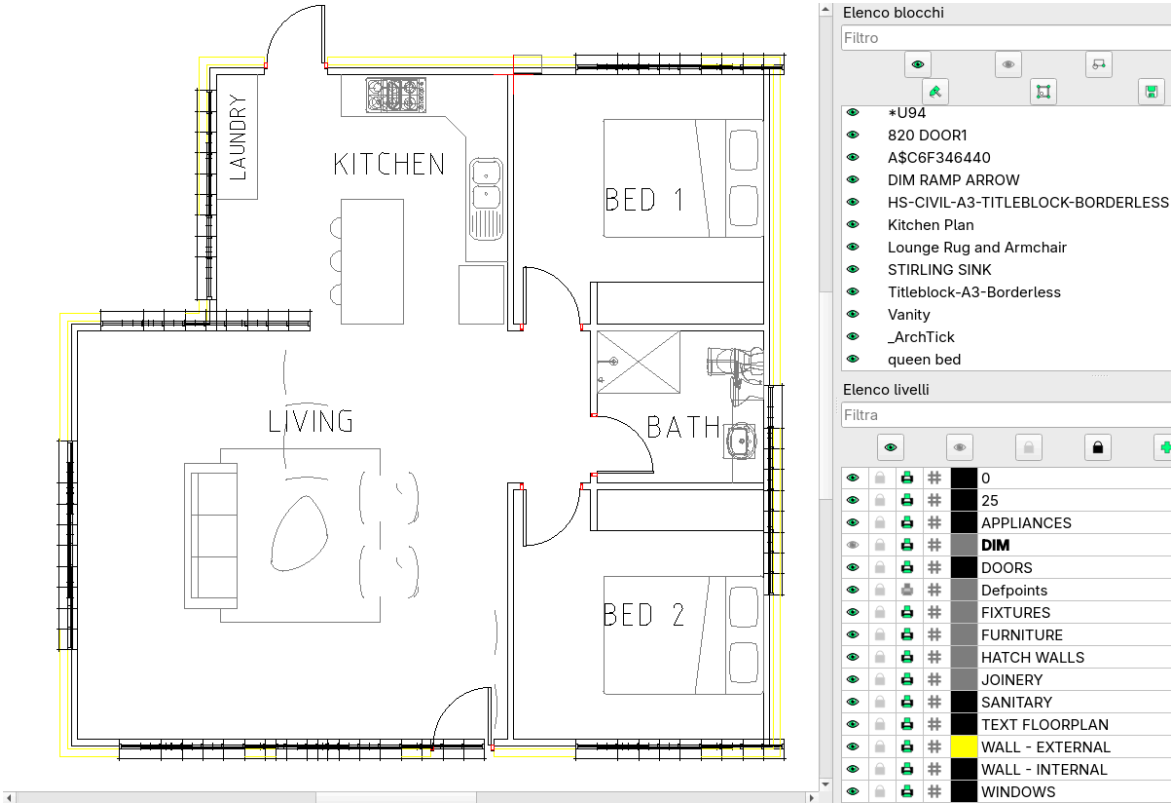
- capire **quali segmenti formano un muro**, dove c'è un'**apertura** e ricostruirla
- capire quali **blocchi** sono porte o finestre e dove posizionarli

### Difficoltà

- Un CAD contiene solo **geometria** (linee, archi, polilinee, blocchi), non la **semantica** dell'edificio
- Un autore può utilizzare una convenzione diversa e rappresentare gli elementi in modi differenti.
- **Layer** con nomi diversi, anche in lingue diverse
- **Rumore e imperfezioni** da rimuovere o correggere
- **Unità di misura** diverse
- Alterazioni nella **conversione DWG → DXF**

# Passo 0: Preprocessing

L'obiettivo è pulire il disegno e renderlo più **consistente**




Modello di partenza: disegno, layer e blocchi

### 1. Nomi

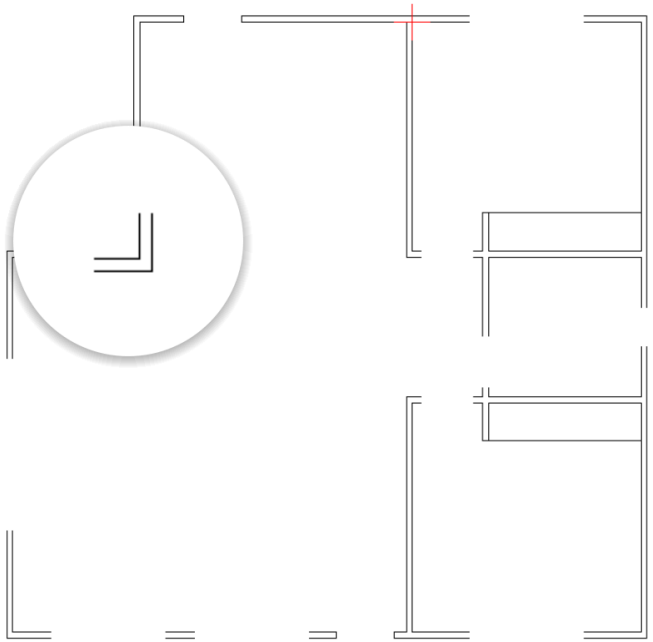
Layer e blocchi rinominati: `A-WALL` `A-DOOR` `A-WINDOW` `BLOCK-DOOR` `BLOCK-WINDOW`

### 2. Pulizia

Rimossi gli elementi inutili o problematici (es. `WALL-EXTERNAL`)

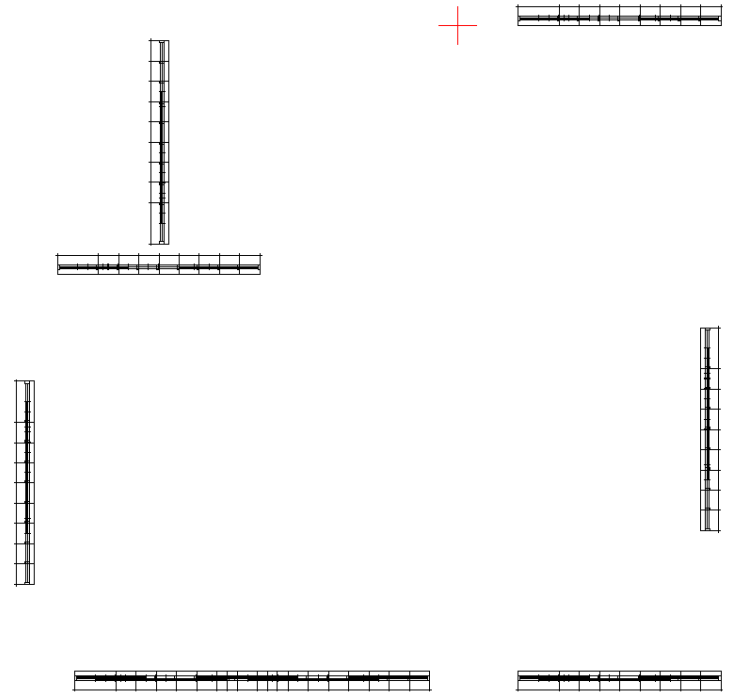
### 3. Correzione

Contorni dei muri chiusi e blocchi delle finestre ripuliti



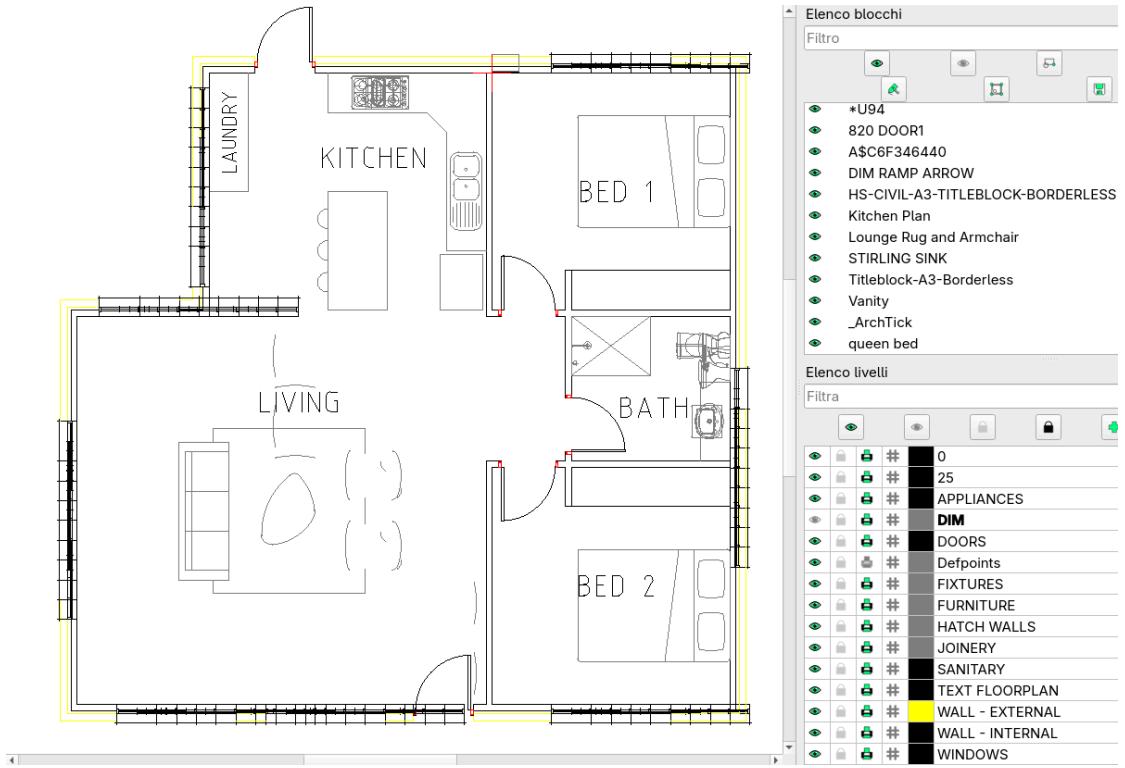
## Muri

La geometria appare rotta

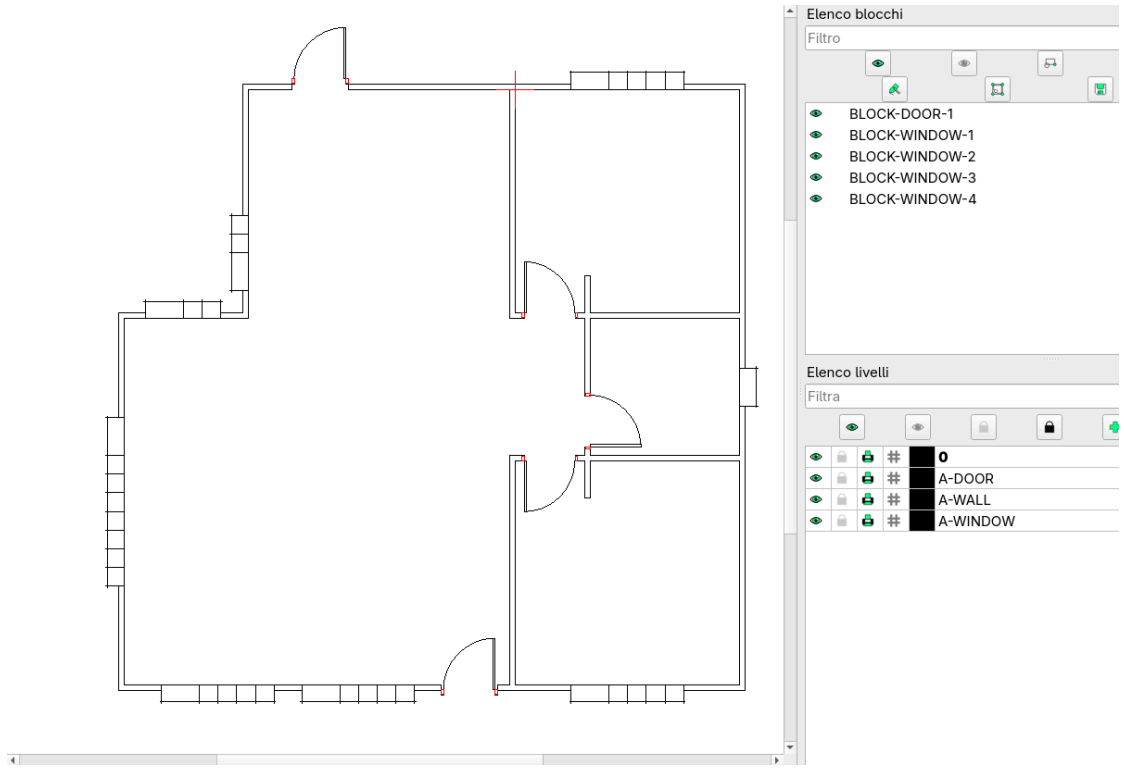


## Finestre

Una finestra può contenere fino a un centinaio di vertici




Prima




Dopo: layer e geometria corretti

# Passo 1: Parsing

### Primitive DXF

`LINE`, `POLYLINE`, `ARC`, `CIRCLE`... raggruppate in **layer** e **blocchi** (componenti riutilizzabili, usati per porte e finestre)

### Obiettivo

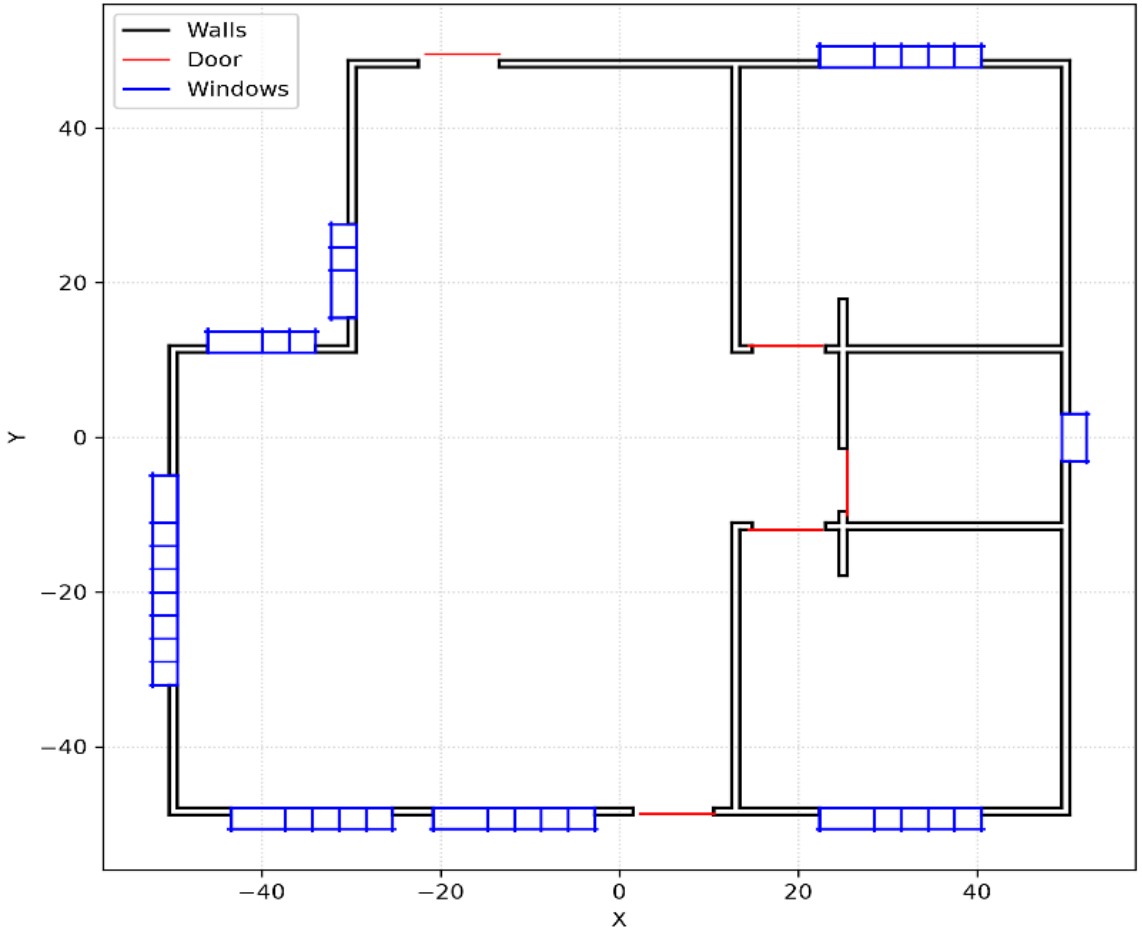
Estrarre muri, porte e finestre, collassando tutte le primitive sottoforma di **segmento**:

$S =< \mathbf{p_0}, \mathbf{p_1}, \mathrm{TAG} >$

$TAG = \{\mathrm{WALL}, \mathrm{DOOR}, \mathrm{WINDOW}\}$

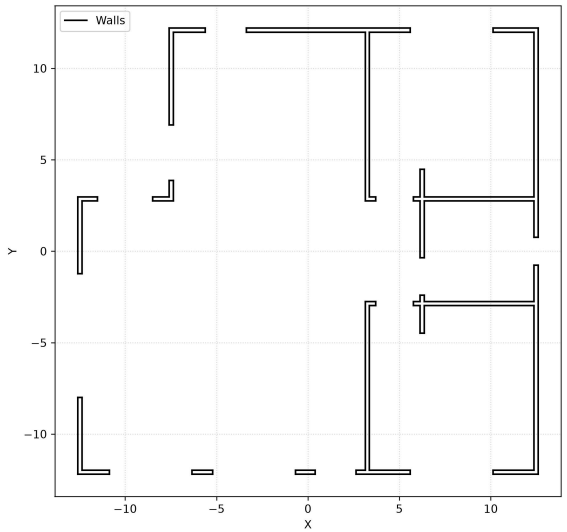
### Risultato

**3 insiemi distinti** di segmenti: muri, porte, finestre

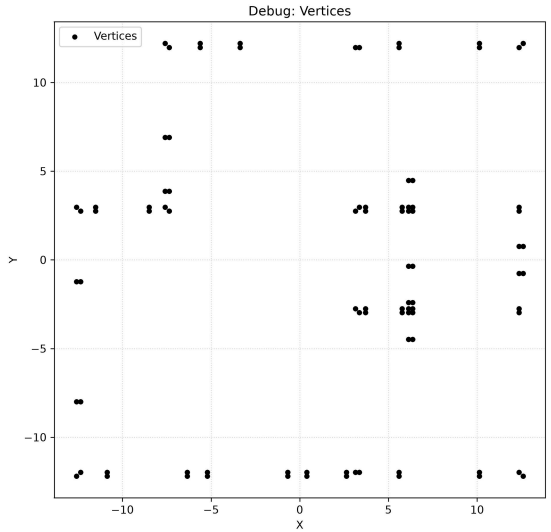



Risultato del parsing

# Passo 2: Collasso dei vertici

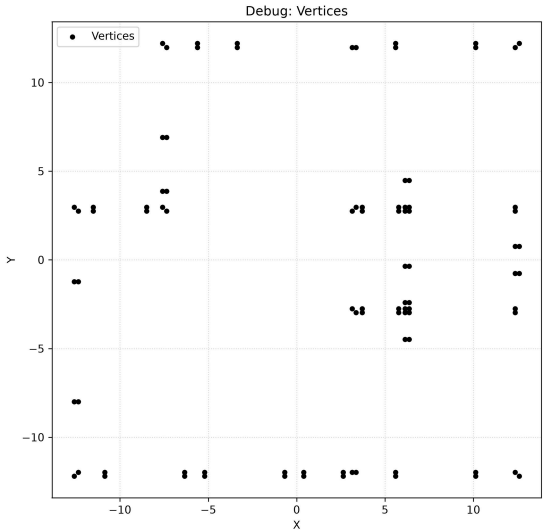



Input: segmenti dei muri




**Prima**

176 vertici




**Dopo**

88 vertici (50% in meno), SSIM: 0.999

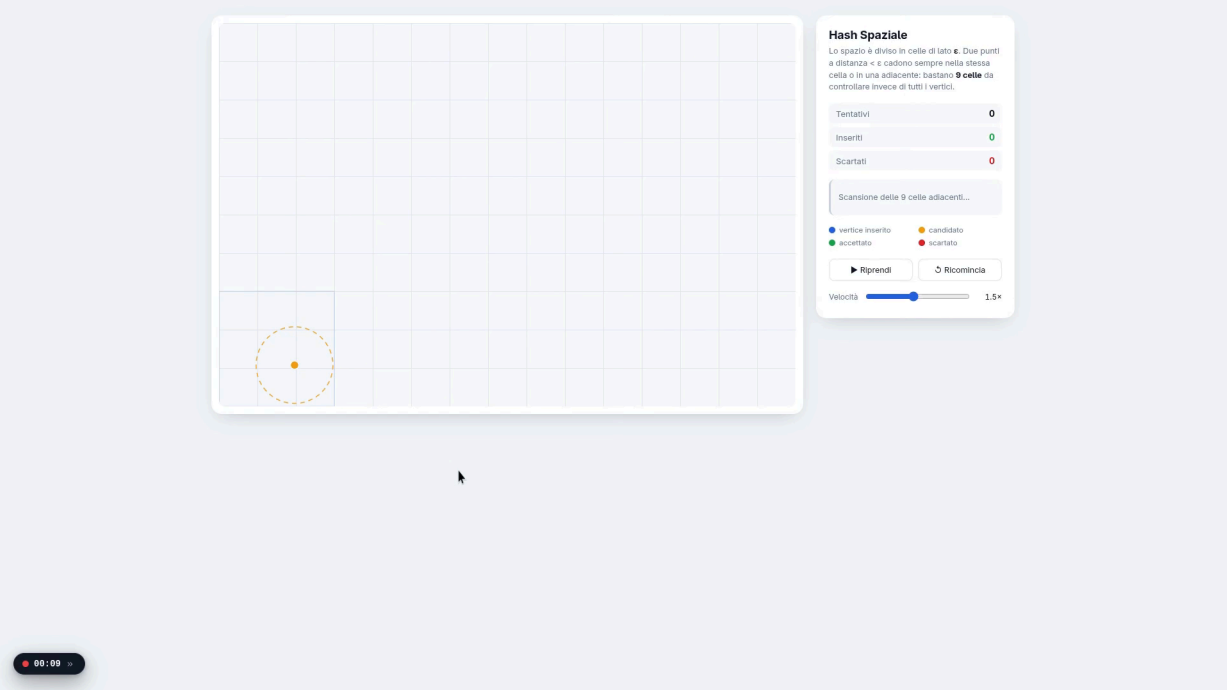
## Problema

Vertici **duplicati**, sovrapposti o così vicini da essere indistinguibili

## Obiettivi

1. **Ridurre** il numero di vertici da elaborare
2. Rappresentare la geometria come **grafo** con vertici univoci

# Hash spaziale




## Griglia

Spazio 2D diviso in **celle di lato fisso**: ogni punto cade in una cella, in base alle sue coordinate

## Ricerca 3×3

Con cella pari al raggio $r$, due punti a distanza $< r$ stanno in una cella adiacente: basta cercare nelle **3×3 celle**

## Regola

$\varepsilon$ = raggio di ricerca = lato della cella. Se esiste un vertice a distanza $< \varepsilon$ il nuovo viene **scartato**, altrimenti **inserito**. Qui $\varepsilon = 10^{-2}$ (da tarare per ogni disegno)

## Risultato

Vertici univoci $V$ e archi $E$: un **grafo non connesso** $G = \langle V, E \rangle$

# Passo 3: Ricostruzione porte




**Problema**

Il grafo appena creato contiene solo i **muri**. I segmenti delle porte collegano due vertici di muro, ma non sappiamo quali

**Idea**

Per ogni segmento rosso (porta) prendo i due estremi e, con la struttura spaziale, cerco i vertici di muro più vicini: i punti di giunzione ($C$, $D$)

**Risultato**

Nuovo edge $e = \langle C, D, \mathrm{DOOR} \rangle$

Nuovo arco (tag `DOOR`) tra C e D, i vertici dei muri più vicini agli estremi della porta

# Passo 3: Ricostruzione porte (2)




Inserimento del secondo edge tra A e B

**Problema**

Un solo edge non basta per ottenere un **poligono chiuso**: serve un secondo edge tra $A$ e $B$

**Come trovare A e B**

Calcolo il versore $\vec{CD}$. Tra i vicini di $C$ ($F$, $A$) scelgo quello **ortogonale** a $\vec{CD}$, sfruttando il prodotto scalare tra versori

**In pratica**

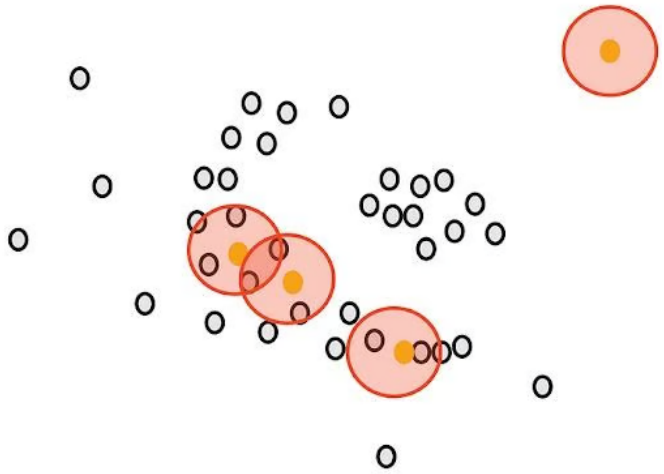
$\vec{CD} \cdot \vec{CF} = -1,\ \vec{CD} \cdot \vec{CA} = 0 \rightarrow$ **A**

$\vec{CD} \cdot \vec{DH} = -1,\ \vec{CD} \cdot \vec{DB} = 0 \rightarrow$ **B**

**Risultato**

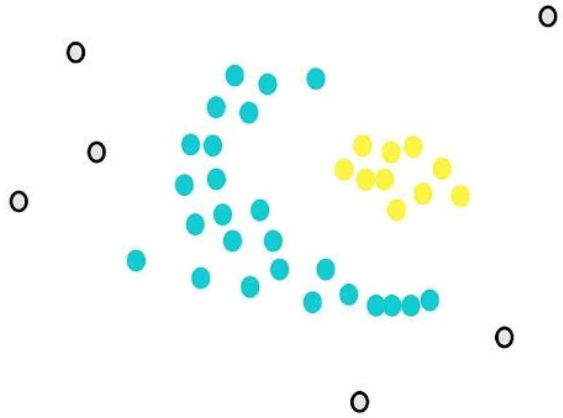
Nuovo edge tra $A$ e $B$. Nessun vertice nuovo inserito.

# Passo 3: Ricostruzione finestre (clustering DBSCAN)



Punti campionati dai segmenti

→



Cluster individuati

**Perché**

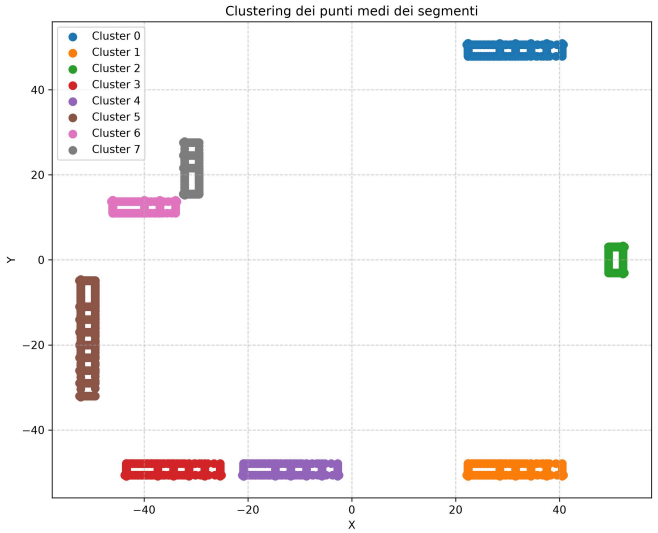
Ogni finestra ha decine di segmenti e cambia da modello a modello: serve un approccio generale

**Parametri**

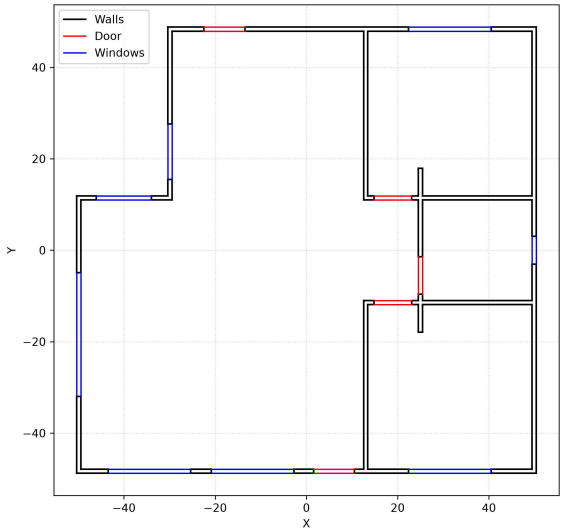
Raggio $r$ (da tarare per ogni disegno) e $m = 2$: se nell'intorno di $p$ ci sono almeno 2 punti, $p$ è un **core point**

**Attenzione a $r$**

**Troppo grande**: cluster distinti si fondono. **Troppo piccolo**: frammentazione




Cluster e bounding box di una finestra




Grafo connesso con porte e finestre ricostruite

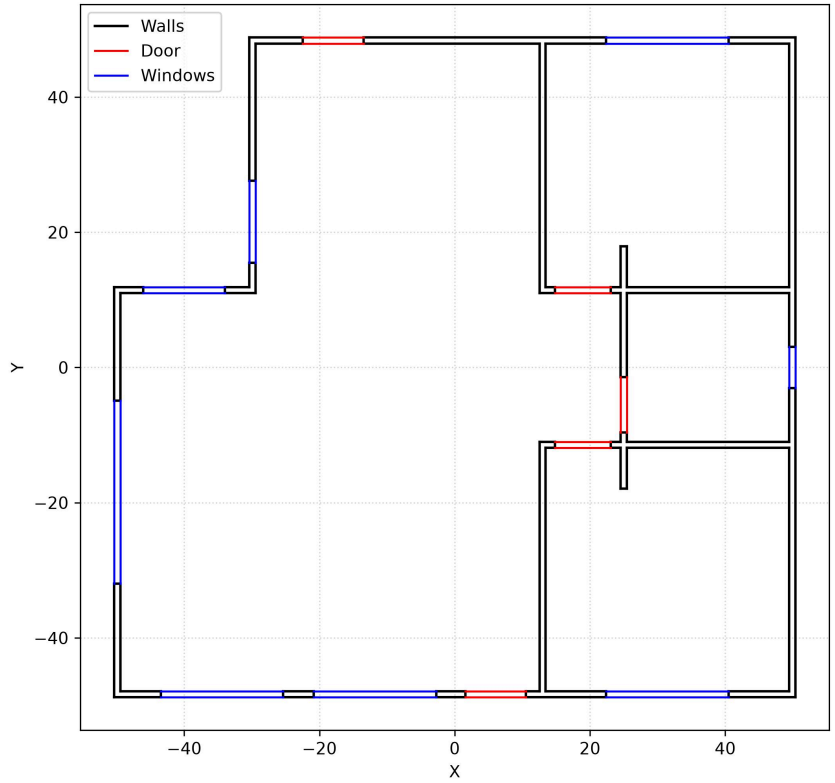
## Procedura

**Bounding box** del cluster → **lato lungo** come segmento di giunzione → stessa procedura delle porte, con $\mathrm{TAG} = \mathrm{WINDOW}$

## Importante

**Nessun nuovo vertice**: sono stati aggiunti solo nuovi edge

# Passo 4: Estrazione delle facce




Grafo dopo la ricostruzione dei varchi

**Input**

Grafo planare con vertici univoci ed edge con TAG (`WALL`, `DOOR`, `WINDOW`)

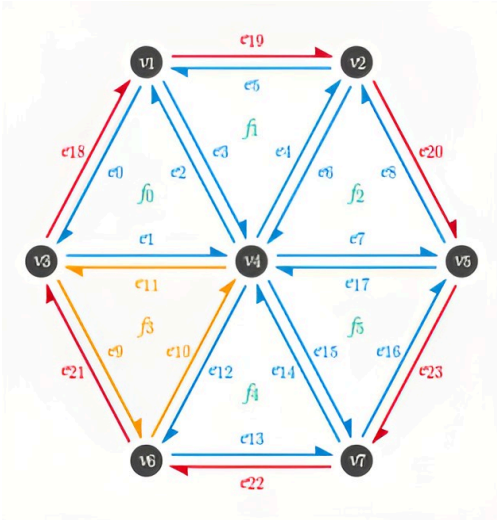
**Obiettivo**

Estrarre le facce: cicli chiusi di edge, cioè **poligoni 2D**

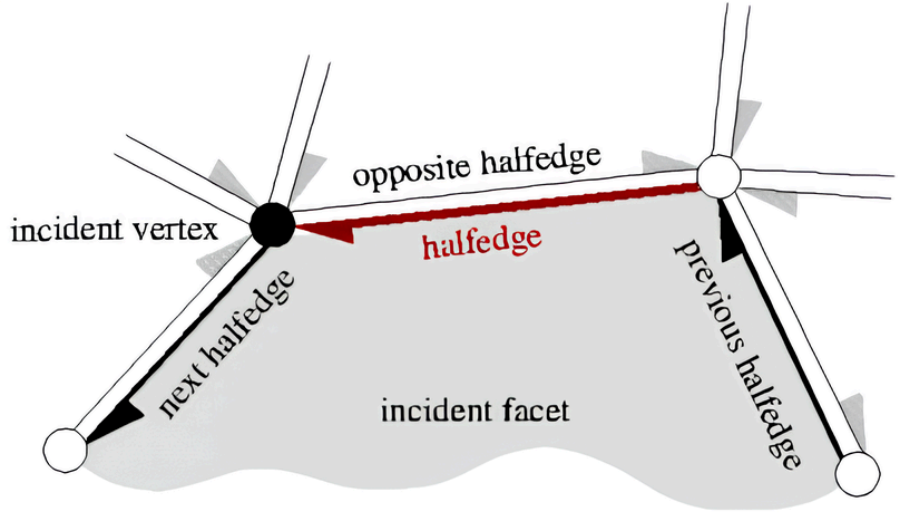
**Problema**

Il grafo dice chi è collegato a chi, ma non l'ordine degli edge attorno a un vertice: serve una struttura che renda esplicita la **connettività**

# La struttura Half-Edge




Due half-edge opposti per ogni edge




Half-edge in CGAL

### Ogni half-edge conosce

La **faccia** sul suo lato, il **vertice** a cui punta, il **twin** (opposto) e il **next** lungo la faccia

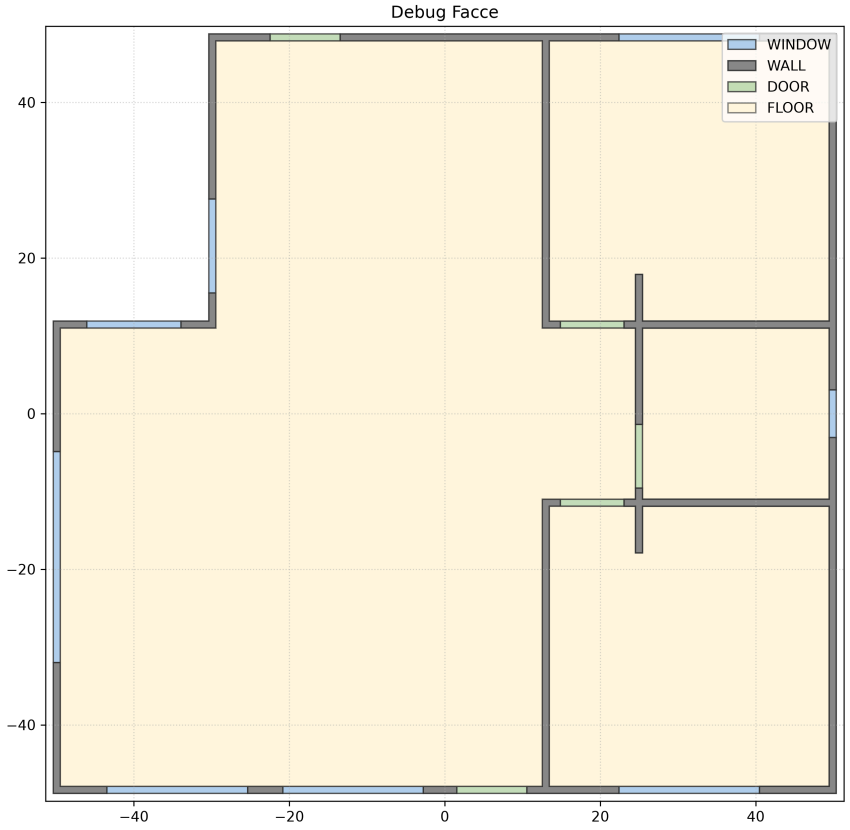
### Percorrendo i next

Si ottiene un **ciclo chiuso**: il contorno di una faccia

### CGAL Arrangement

Vertici, half-edge e **lista di facce**. I **tag** restano su ogni half-edge e sul suo twin. Risultato: **poligoni 2D**

# Classificazione delle facce




Facce classificate: stanze, porte e finestre

**Perché**

Serve per l'**estrusione differenziata**: ogni tipo di faccia ha un'altezza diversa

**Porta**

2 edge `DOOR` + 2 edge `WALL`

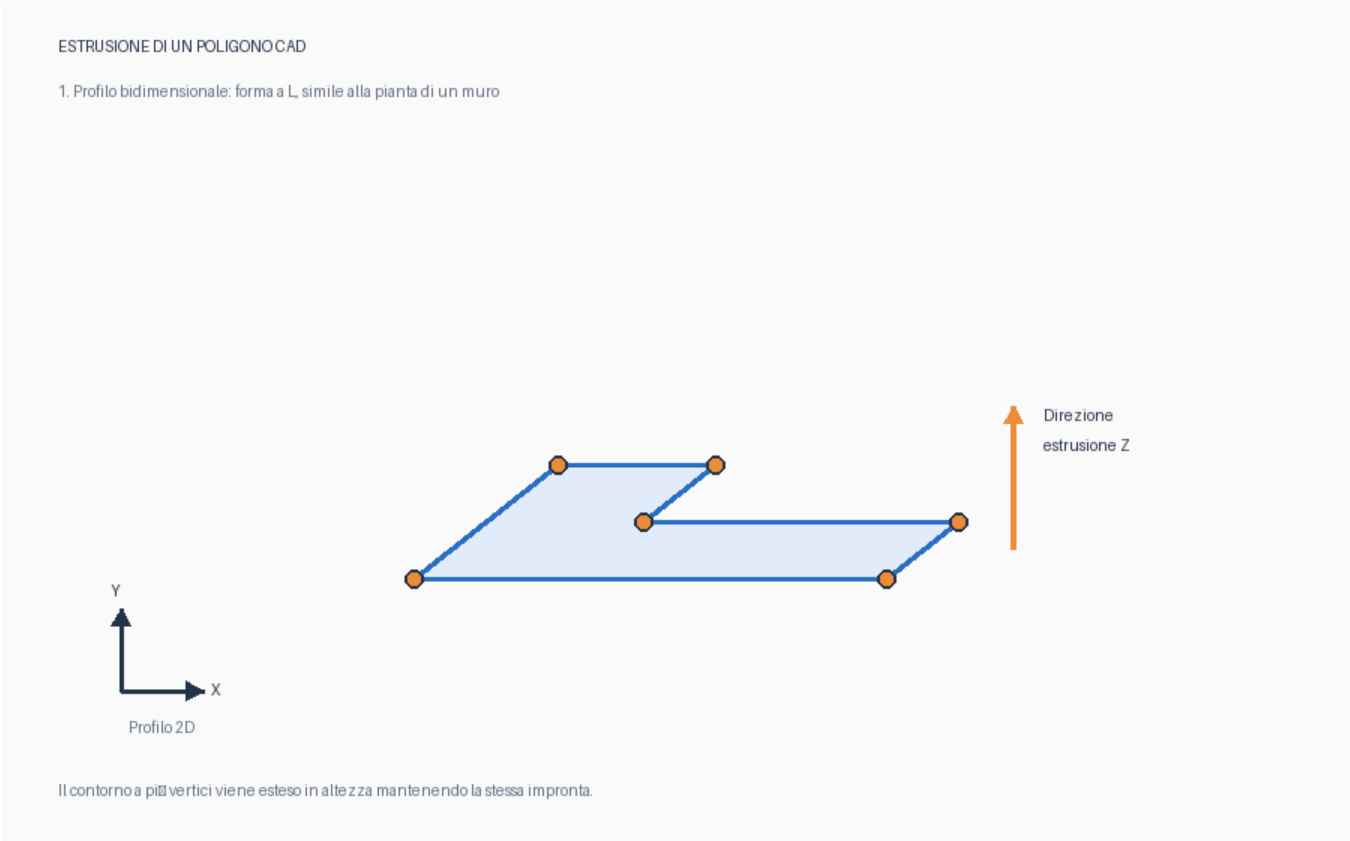
**Finestra**

2 edge `WINDOW` + 2 edge `WALL`

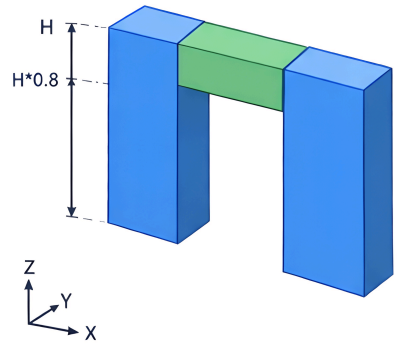
**Muri**

Solo edge `WALL`

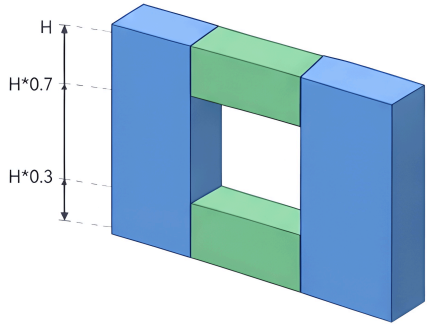
# Passo 5: Estrusione differenziata




Da poligono 2D a solido 3D

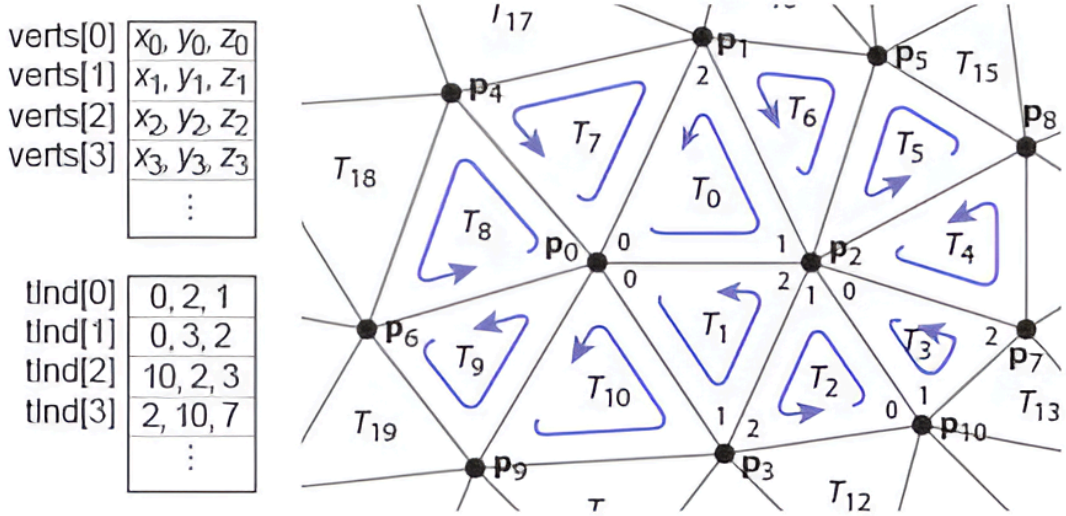



Porta




Finestra

# Rappresentazione di una mesh




Indexed triangle mesh: buffer dei vertici e buffer degli indici

**Mesh triangolare**

Rete di triangoli connessi, che non deve contenere buchi

**Vertici**

Salvati in un **vertex buffer**.

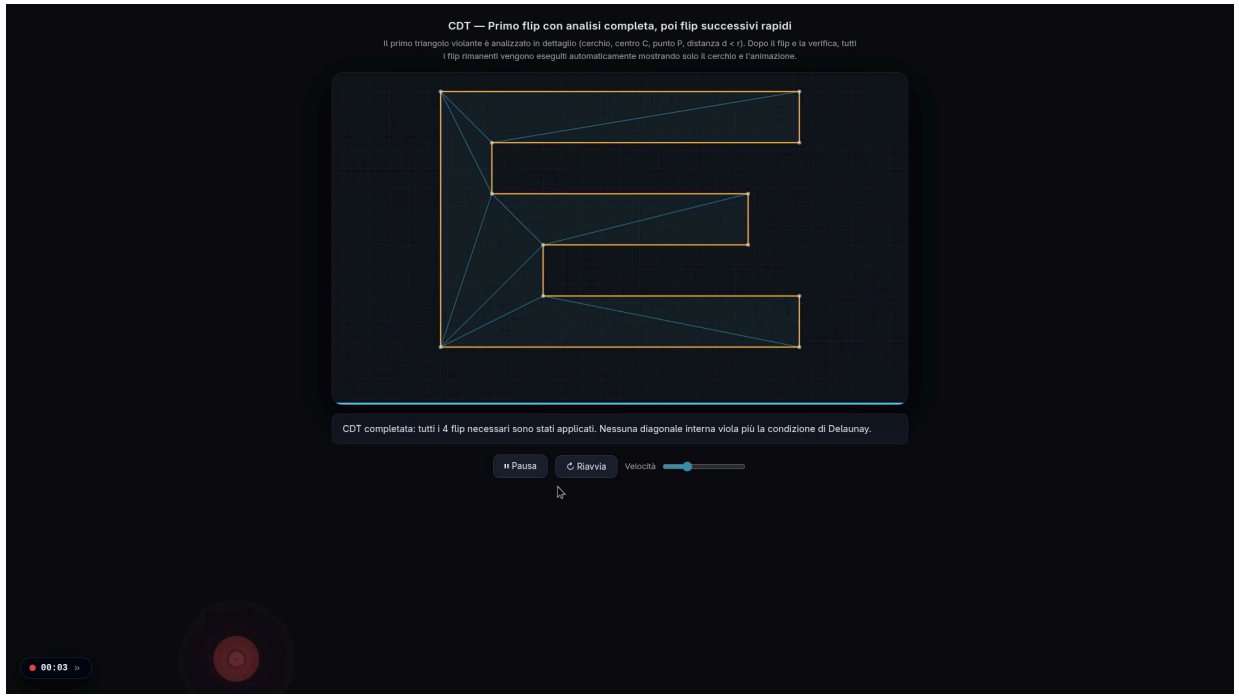
Per ogni vertice vengono calcolati e memorizzati i seguente attributi: posizione, normale, texture coordinate.

**Indici**

Ogni triangolo richiama i suoi vertici gli indici.

L'insieme degli indici sono salvati in un **index buffer**.

# Passo 6: Triangolazione di Delaunay




Flip degli archi che violano la condizione di Delaunay

## Problema

Scomporre un poligono semplice in triangoli, usando l'algoritmo **Constrained Delaunay Triangulation**

## Flusso

1. **Triangolazione iniziale** (Ear Clipping, Sweep Line). Qui: Poly2Tri con Sweep Line
2. **Flip di Lawson** solo sugli archi interni che violano la condizione di Delaunay

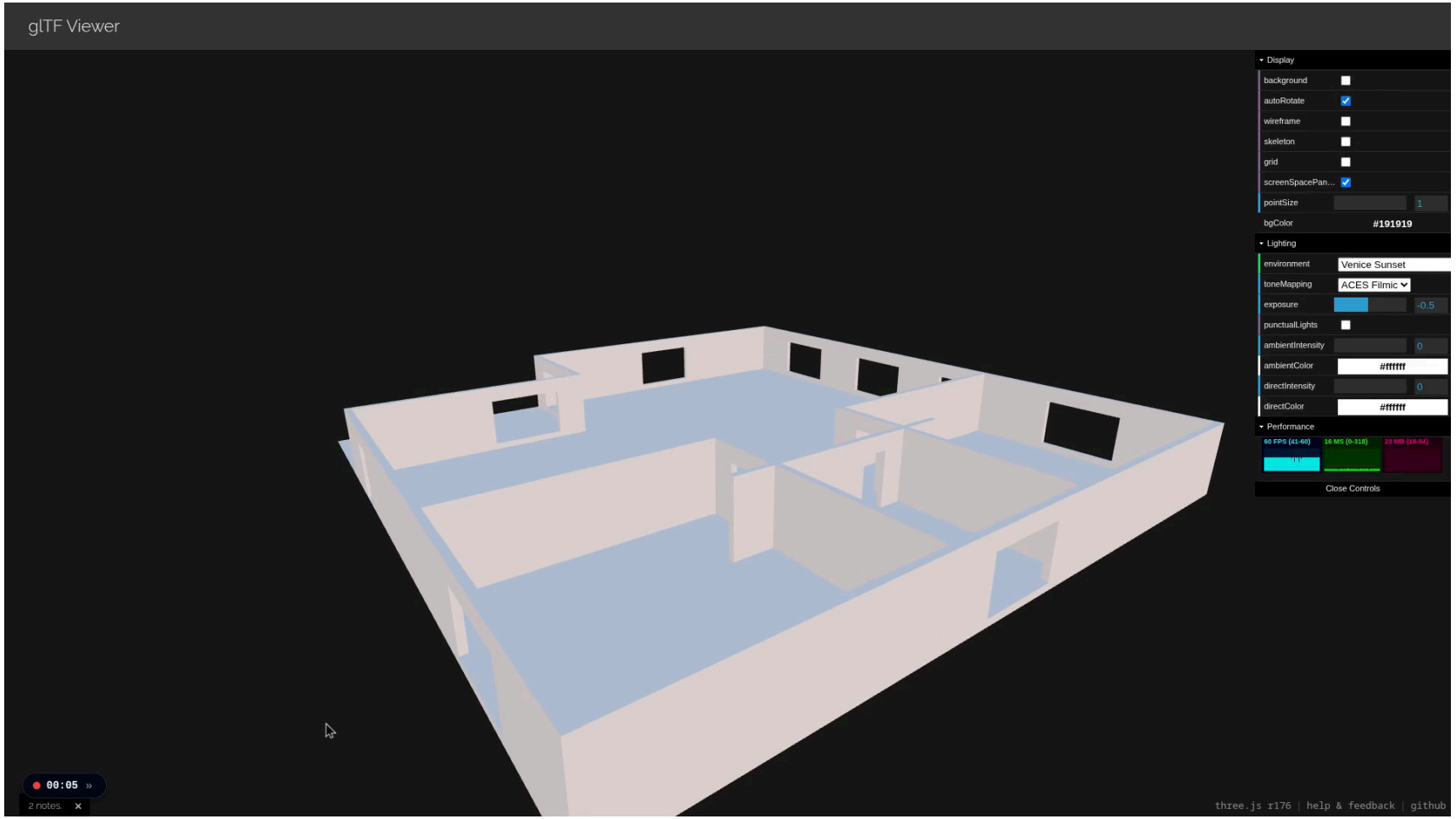
## Condizione di Delaunay

Nessun punto dentro il cerchio circoscritto di un triangolo

## Vantaggi

Evita i **triangoli degeneri** e permette di imporre segmenti obbligatori

# Mesh Preview




Mesh 3D generata dalla pipeline (prima dell'inserimento degli asset)

# Passo 7: Esportazione

### Problema

La mesh esportata in **GLTF** ha dei buchi vuoti a ridosso di porte e finestre: servono asset 3D

### Instancing in Blender

Una mesh ripetuta non duplica i vertici: sono allocati **una sola volta** e condivisi. Cambiano solo la matrice di trasformazione e il materiale

### Asset nella pipeline (scartato)

- Geometria **duplicata**: 10 porte, 10 copie
- Cambiare asset o orientamento = **rieseguire la pipeline**
- **Materiali originali** da preservare

### Approccio a placeholder (scelto)

La pipeline esporta solo dove inserire l'asset ( `center` ), l'**orientamento** ( `rotate_z` ) e la **scala**. Il caricamento avviene in Blender

**Pipeline**
genera la mesh

→

`mesh.gltf`
`placeholders.json`

→

**Blender**
carica gli asset

# Rendering in Blender: approccio ibrido

### GLTF: solo geometria

Posizioni, normali e coordinate texture, nessun materiale. Materiali, asset, luci, camera ed environment map sono definiti in **Blender**: posso cambiare i materiali senza rieseguire la pipeline.

### Solo script Python? Scartato

Camera e luci (posizione, rotazione, intensità) sarebbero **hard-coded**: molto scomodo e poco intuitivo

### Script Python ( `bpy` )

Operazioni ripetitive e deterministiche

1. Importa la **mesh GLTF**
2. Definisce i **materiali** (albedo, normal, roughness...) per pavimento e muri
3. Carica l'**environment map**
4. Legge i **placeholder** e carica gli **asset**
5. Crea la scena iniziale `.blend`

### GUI di Blender

Camera e luci impostate navigando la scena, in modo semplice. Aggiungere arredi (divano, TV, lampada) è immediato

# Rendering con Cycles

**Cycles**: path tracer *physically-based*

**Risoluzione**

**1280×720**

**Campioni**

**128 spp**

con denoiser **OpenImageDenoise**

**Tempo**

**3-7 min**

per immagine, in base alla complessità

Su **CPU** AMD Ryzen 5 2600X

# Risultati







